

1 Intro

Check out my code. It's very good.

2 part 1

a) $dt = 0.005$. The robot acts like a pendulum with no damping.

b) $dt = 0.5$. The robot flails around wildly.

The total energy should be equivalent to the sum of the total energy for each rigid body (aka all the moving links). Total energy of each rigid body can be computed using $T = PE + KE$, where PE is the potential energy of the rigid body and KE is the kinetic energy of the rigid body. PE should be mgh and KE should be $1/2mv^2 + 1/2I\omega^2$

ref: link

```
g = np.array([0, 0, -9.81]).T
tt = 10.0
dts = [0.005, 0.5]
```

3 part 2

a) damping = 2.0

b) damping = -0.01

If the damping is too large of a positive value, then the damping torques will actually be larger in magnitude than the other torques, thereby causing the joints to move in the opposite direction as opposed to simply slowing down the forward momentum. Because the damping torques scale with joint velocity, with shorter simulation timesteps, we would expect the robot to come to a stop without reversing direction, since the damping torques would asymptotically converge onto all zeros as the joint velocities reduced.

```
g = np.array([0, 0, -9.81]).T
tt = 10.0
dt = 0.01
dampings = [2.0, -0.01]
```

4 part 3

a) stiffness = 5.0

The springy string is acting like a perfect spring, meaning that it dissipates no energy. The robot also dissipates no energy with damping = 0.0. Therefore we would expect the robot to oscillate about the spring position, similar to a pendulum affected by gravity.

I expect the total energy of the system to remain constant over time since there's no damping.

If the spring stiffness is too high, we'd expect to see numerical instability. The tip wrench would be some very large value, thereby inducing very large joint accelerations, thereby inducing very large joint velocities, thereby inducing erratic and chaotic joint position values.

b) damping = 2.0, stiffness = 5.0

a)

```
g = np.zeros([1, 3])
tt = 10.0
dt = 0.01
damping = 0.0
restLength = 0.0
stiffnesss = [5.0] # , 10.0]
```

b)

```
damping = 2.0
stiffness = 5.0
```

5 part 4

Done. see video.

```
tt = 10.0
dt = 0.01
damping = 2.0
stiffness = 5.0
g = np.zeros([1, 3])
restLength = 0.0
```